

Westminster Chimes and Carillon Bells 8.1

Comprehensive application guide for the FMX portable edition.

Last documented: 2026-07-04

Primary project folder: C:\New Delphi Projects\Carillon8_1- Portable - FMX

1. Purpose

Westminster Chimes and Carillon Bells is a portable FireMonkey desktop application for scheduled church-bell playback and related administration. The application combines:

- a playlist editor and daily scheduler
- recurring and date-bound seasonal playback rules
- random music directory rotation
- audio output and mute control
- optional email and morning-report delivery
- portable deployment from a thumbdrive without a Windows installer

The system is designed to run continuously, generate a daily play schedule, and play sound files automatically at configured times.

2. High-Level Architecture

The application is a classic Delphi FMX desktop app with several cooperating forms and a SQLite database.

Main source units:

Unit	Role
Carillon.dpr	program entry point, single-instance lock, splash screen, form creation
Playlist.pas	main operational form, playback coordination, grouped grid, theme propagation, help, shutdown logic
SettingsForm.pas	global settings, organization name, logging, liturgical silence settings
GroupForm.pas	seasonal group maintenance and holiday date recalculation
RandomDirectory.pas	seven-slot random-music directory rotation editor
email.pas	email settings, test mail, morning report generation and dispatch
ScheduleManager.pas	schedule-building logic and playback-entry generation
LogManager.pas	application log path, ISO timestamp writing/parsing, and log trimming
RandomDirectoryManager.pas	random directory slot loading and rotation resolution
SilenceManager.pas	reusable silence-window checks for playback suppression
EasterCalculator.pas	shared Easter-date calculation used by groups/settings
CarillonTheme.pas	single source of truth for theme definitions and shared visual rules
ClockUnit.pas	analog clock window
CarillonSplash.pas	startup splash screen

AudioManager.pas	Windows audio endpoint volume integration
Overlay.pas	translucent overlay form used behind modal dialogs
MMDevAPI.pas	Windows audio COM interface declarations
PowerHoldSimple.pas	helper used by the app but not a primary operational form

3. Startup Sequence

Startup is controlled by Carillon.dpr.

Sequence:

1. The program attempts to open and exclusively lock Carillon.lock in the executable folder.
2. If the lock cannot be acquired, the app shows a message and exits, preventing multiple running instances.
3. Application.Initialize runs.
4. The splash screen form (CarillonSplash) is shown for roughly 2.5 seconds.
5. The main and support forms are created:
 - TfmDailyPlayList
 - TfrmOverlay
 - TGroups
 - TfmClock
 - TSettingsMain
 - TfrmEmailSettings
 - TfrmRandomDirectory
6. Application.Run enters the FMX message loop.

Important note:

- Memory-leak popup reporting on shutdown was intentionally disabled in the program file to avoid forced-shutdown leak dialogs during Windows restart/update scenarios.

4. Main Form: Playlist

Playlist.pas is the operational heart of the application.

4.1 Main Responsibilities

The playlist form is responsible for:

- loading the main playlist dataset
- applying and persisting color themes
- coordinating active daily schedule rebuilds
- playing scheduled songs and manually selected songs
- managing the grouped playlist grid
- committing schedule edits through the navigator
- handling random-song substitution
- logging playback through LogManager
- coordinating help display
- updating the runtime status bar
- performing orderly shutdown when the application exits

4.2 Main Data Components

Key runtime components include:

- PlaylistConnection: TFDConnection
- PlaylistQuery: TFDQuery
- plDataSource: TDataSource
- ScheduleTimer: TTimer
- tmRebuildSched: TTimer
- Timer1: TTimer
- KeepUSBAliveTimer: TTimer
- MP3MediaPlayer: TMediaPlayer

4.3 Playlist Grid

The playlist grid is a grouped view built over the dataset. In 8.1 it uses larger, more readable row text and adjusted row height so rows display cleanly without clipped bottom rows.

Features:

- grouped by seasonal group / season
- collapsible group headers
- startup state can be open or closed, controlled by StartPlaylistGroupsClosed
- Open All and Close All
- alternating row colors
- two-column presentation:
- song name
- duration
- theme-aware header, selection, group, and alternating row colors

The grouped grid is a custom display layer over the dataset, not a native FMX tree grid.

4.4 Scheduling Model

Schedule building is handled by ScheduleManager.pas and stored in a list of TScheduleEntry records.

Current TScheduleEntry fields:

- NumberOfTimesToPlay
- PlaylistRecordNo
- SongPath
- ScheduledTime
- ScheduleEntryId

Important behavior:

- the schedule is built from the playlist table
- schedule entries are filtered by date range and weekday flags
- up to 12 scheduled times can exist per playlist row
- entries are sorted and processed by ScheduleTimer
- scheduled playback no longer relies on long-lived stored dataset bookmarks
- opening Groups, Settings, Email Settings, or Random Music disables and then re-enables the schedule so the schedule is rebuilt after maintenance changes
- playlist schedule edits disable the active schedule only after an actual field change, then rebuild after navigator post
- navigator cancel restores the dataset values shown in the play-time boxes

- group synchronization is a clean overwrite: the group play time is copied to `scheduled_time1` and `scheduled_time2` through `scheduled_time12` are cleared to prevent stale extra plays

4.5 Playback Flow

Normal playback path:

7. Determine the target song.
8. If the song path is a placeholder/random entry, resolve a real file from the configured random directory.
9. Update the now-playing display.
10. Play the media using `TMediaPlayer`.
11. Repeat according to `num_times_to_play`.
12. Log the play event if logging is enabled.

Playback can happen by:

- manual button press
- double-clicking a grid row
- timer-based scheduled playback

4.6 Editing and Navigator Behavior

Editable data screens use `TBindNavigator` controls for posting or canceling database edits. Navigator labels identify common actions such as Add, Delete, Save, Cancel, First, Prior, Next, and Last.

The main play-time fields accept normal AM/PM entries and seconds. For example, 11:59:58 PM is stored as 23:59:58 and displays back with seconds instead of being shortened to 11:59 PM.

4.7 Random Music Resolution

Random-song behavior is handled in `Playlist.pas` by helpers such as:

- `ResolveRandomSongIfNeeded`
- `RetrieveRandSongsDirectory`

These use:

- the current random-music rotation settings
- directory/date configuration from `PL_RAND_DIR_ROTATION`
- file scanning of the selected random directory

4.8 Help and Support Files

The main form opens the help file from:

- `Help\Carillon Bells Help.chm`

The help lookup is executable-folder relative, which is appropriate for portable deployment.

4.9 Logging

Playback logging writes to:

- `logs\CarillonPlayLog.txt`

The code now ensures the log directory exists before writing. Log entries use an invariant ISO timestamp format: `yyyy-mm-dd hh:nn:ss - message`. The same parser is used by log trimming and morning-report email generation.

4.10 Keepalive Behavior

The keepalive timer writes to:

- `data\keepalive.txt`

This acts as a simple “USB alive” / runtime heartbeat file and also ensures the data folder exists.

4.11 Shutdown Handling

The playlist form contains deliberate shutdown hardening.

It centralizes shutdown through `BeginOrderlyShutdown`, which:

- marks the app as shutting down
- disables timers
- stops media activity
- prevents further playback/schedule work during close

This was added to reduce forced-shutdown instability and Windows-restart problems.

5. Settings Form

`SettingsForm.pas` manages system-wide application settings stored in `pl_settings`.

Main responsibilities:

- organization name
- logging status
- silence-window settings
- seasonal silence date/time ranges

Important settings areas:

- `organization_name`
- `form_bgcolor`
- `form_fontcolor`
- `logging_status`
- silence-related dates and times

The settings form uses a navigator for posting or canceling changes. It also drives organization-name values used in splash/reporting and liturgical silence settings used by playback checks.

6. Group Form

`GroupForm.pas` maintains seasonal groups and their associated scheduling windows.

Main responsibilities:

- edit seasonal group rows in the database
- display data through a grid editor model
- recalculate holiday-driven date ranges
- apply current theme colors to the maintenance form
- disable/re-enable the main schedule on open/close so schedule changes are rebuilt

Holiday/date logic in this form supports maintenance of date-driven groups such as:

- Easter
- Thanksgiving
- Memorial Day
- and similar seasonal windows

This form is an administrative maintenance tool, not the live playback engine. When group schedule information is synchronized to playlist rows, stale time slots 2 through 12 are cleared so old manual times cannot cause unwanted extra playback.

7. Random Directory Form

`RandomDirectory.pas` provides the editor and `RandomDirectoryManager.pas` manages the row-based `PL_RAND_DIR_ROTATION` table. The 8.1 screen was redesigned to expose all seven slots in a more compact layout.

Purpose:

- define up to seven random music directories
- define annual from/to dates for each directory slot
- ensure the configuration table always contains the expected row

Important behavior:

- `EnsureSingleRowExists` inserts row `id = 1` when missing
- annual dates are edited as month/day-style recurring values
- changes are saved or canceled with the Random Directory navigator buttons
- directory boxes and date fields are compacted for more efficient use of screen space
- theme settings are loaded from `pl_settings`
- opening and closing this screen forces a schedule rebuild when needed

This form feeds the playlist form's random-song substitution logic.

8. Email Settings Form

`email.pas` manages email configuration and outbound email features.

Main responsibilities:

- email enable/disable
- SMTP host/port/user credentials
- app password storage
- test-recipient maintenance
- general recipients
- morning-report recipients
- sending test mail
- sending morning reports
- test morning-report sending

Important technologies:

- Indy SMTP components
- OpenSSL support DLLs:
 - `libeay32.dll`
 - `ssleay32.dll`

Important database/settings interaction:

- reads email settings from `pl_settings`
- reads organization name from `pl_settings`
- reads form theme colors from `pl_settings`

9. Morning Report

Morning-report generation is handled from `email.pas`.

It gathers:

- CPU, memory, and disk metrics
- playlist counts
- total playlist duration
- broken/missing media files
- playback log excerpts

Important 8.1 behavior:

- the report reads `logs\CarillonPlayLog.txt` through `LogManager.CarillonLogFilePath`
- log timestamps are parsed with the same ISO parser used by log trimming
- startup, error/warning, maintenance, yesterday count, and recent-play sections are based on that shared log format

10. Splash Screen

`CarillonSplash.pas` shows startup branding and current organization information.

It loads:

- `ORGANIZATION_NAME`
- `RAND_SONGS_DIRECTORY`

from `PL_SETTINGS`.

Visible splash elements:

- hero image
- version label
- organization-name panel

The org-name panel is now centered by runtime layout logic rather than by fake space padding in the label text.

11. Clock Form

`ClockUnit.pas` draws an analog clock over a background image.

Responsibilities:

- render hour/minute hands
- update once per second
- resize and center the form for the active display resolution

This is a secondary display utility window.

12. Audio Control

`AudioManager.pas` wraps Windows endpoint audio volume control.

Responsibilities:

- discover the default audio render endpoint
- read the current master volume
- sync the UI trackbar to system volume

- set mute/unmute
- update master volume from the trackbar

This uses COM and the Windows audio endpoint API through `MMDevAPI.pas`.

13. Database Overview

The application uses SQLite through FireDAC.

Primary database path:

- `.\databases\carillon.db`

For the portable runtime copy, launch from the target drive root, such as `E:\`, so relative database, help, log, and media paths resolve correctly.

Known tables from source usage:

- `playlist`
- `pl_settings`
- `PL_RAND_DIR_ROTATION`
- `SeasonalGroups`

13.1 `playlist`

Stores the playable items and scheduling fields.

Observed fields include:

- `song_name`
- `song_duration`
- `playlist_name`
- `season`
- `play_date_from`
- `play_date_to`
- `play_monday` through `play_sunday`
- `scheduled_time1` through `scheduled_time12`
- `num_times_to_play`

13.2 `pl\_settings`

Stores application-wide settings including:

- organization name
- random songs directory
- form background color
- form font color
- logging status
- email settings
- morning-report recipients
- silence settings

13.3 `PL\_RAND\_DIR\_ROTATION`

Stores rotating random-music directories and annual date windows as one row per slot (`slot_no`, `rand_music_dir`, `frm_date`, `to_date`). Slots 1 through 7 are exposed in the Random Music screen; slot 1 is the default fallback folder.

13.4 `SeasonalGroups`

Provides group names, date windows, and group play times used to update playlist records and populate the seasonal-group selection UI. During group synchronization, playlist rows for that group receive the group date range and primary play time, while old secondary schedule times are cleared.

14. Theme and Visual System

The app uses a theme palette system defined in `CarillonTheme.pas`, which is the single source of truth for theme names, colors, and shared palette rules.

Each theme carries:

- menu item name
- theme display name
- accent color
- background color
- font color
- surface color

Theme application currently covers:

- form fill
- labels
- buttons
- inputs
- grouped playlist grid
- hero/banner graphics
- status bar
- schedule display dialogs and table headers

The main theme controller lives in the playlist form and propagates colors to other forms.

15. Portable Deployment Model

This application is intended to be portable.

Expected runtime package layout:

- `Carillon.exe`
- `start_carillon.vbs`
- `start_carillon.bat`
- `databases\carillon.db`
- `Help\Carillon Bells Help.chm`
- `logs\`
- `data\`
- `Songs\`
- `Music\`
- `Random_songs\`
- `Random_songs-2\`
- `libeay32.dll`
- `ssleay32.dll`

Important deployment characteristics:

- no installer required

- no registry-based installation requirement
- intended to be zipped and deployed to removable media
- startup can be driven by the .vbs launcher for a cleaner boot experience than the .bat alone
- the runtime executable should be launched with the portable drive root as the working directory, for example E:\Carillon.exe with working directory E:\

16. Runtime Files and Operational Artifacts

Files created or used at runtime include:

- Carillon.lock
- logs\CarillonPlayLog.txt
- data\keepalive.txt

These should be considered part of the live runtime environment, even if not required to be pre-populated in a fresh package.

17. Operational Features Summary

Core operational features:

- scheduled playback by date and weekday
- repeated playback count per entry
- grouped playlist browsing
- seasonal-group maintenance
- random directory rotation
- email notification/testing
- morning reports
- runtime help file access
- system audio control
- optional analog clock display

18. Known Maintenance Notes

These are important for future developers/maintainers.

13. Some legacy component names from earlier ADO-era development remain, even though FireDAC is used.
14. Use MSBuild for validation rather than raw dcc32.
15. Launch runtime tests from the portable drive root so the SQLite database and relative folders are found.
16. Theme definitions are centralized in CarillonTheme.pas; future visual work should continue using that single source of truth.
17. The schedule builder lives in ScheduleManager.pas; future scheduling changes should be made there when practical.

19. Recommended Future Documentation Additions

This guide is a top-to-bottom operational overview. For deeper maintenance, consider adding:

- a field-by-field database schema reference
- a theme palette reference with screenshots
- a deployment checklist with exact thumbdrive ZIP contents
- an operator quick-start guide
- a troubleshooting guide for email, media paths, and schedule generation

- a validation checklist for seasonal group overlap and schedule rebuild testing

20. Quick Reference

Key files:

- main program: `Carillon.dpr`
- main operational form: `Playlist.pas`
- settings: `SettingsForm.pas`
- email: `email.pas`
- random rotation: `RandomDirectory.pas` and `RandomDirectoryManager.pas`
- seasonal groups: `GroupForm.pas`
- splash: `CarillonSplash.pas`
- clock: `ClockUnit.pas`
- schedule manager: `ScheduleManager.pas`
- log manager: `LogManager.pas`
- random directory manager: `RandomDirectoryManager.pas`
- silence manager: `SilenceManager.pas`

Key folders:

- database: `databases\`
- help: `Help\`
- logs: `logs\`
- heartbeat/runtime data: `data\`
- sound libraries: `Songs\`, `Music\`, `Random_songs\`, `Random_songs-2\`

This document is intended to give a future maintainer or operator enough context to understand how the application is organized, how it runs, and where to begin when changing behavior.